operational_endpoints

Helix OTA — Operational REST API: Audit, Telemetry-Read & Device-Group Endpoints

Field	Value
Revision	1
Created	2026-06-08
Last modified	2026-06-08
Status	active (deferred-phase specification — not in the 1.0.0-MVP built surface)
Status summary	Prose specification of three **operational** `/api/v1` API areas deferred out of 1.0.0-MVP and routed by `additions_synthesis.md` §8 (gap register) / §9 (new work items): **(A) Audit subsystem** — a mutating-action audit middleware that writes the canonical `audit_logs` table (G3), plus a `GET /audit` query endpoint and the `AppendAudit` repository method; **(B) Telemetry reads** — `GET /devices/{deviceId}/telemetry` per-device history and `GET /telemetry/overview` fleet aggregates over the already-built `telemetry_events` ingest path (G4); **(C) Device-group CRUD + membership** over the existing `device_groups` / `device_group_members` tables (G5). This document extends, and is subordinate to, `endpoints.md`: all conventions (base path, error envelope, RBAC roles, transport/compression, pagination) are inherited from there and only the deltas are stated here.
Issues	These endpoints are **NOT implemented** in `server/internal/api` as of this revision: there is no audit

Field	Value
	middleware, no telemetry read side, and no group-CRUD routes (`additions_synthesis.md` §8 G3/G4/G5 — "PARTIAL", confirmed against the built handler set this pass). The `audit_logs`, `device_groups`, `device_group_members`, and `telemetry_events` **tables already exist** in `database/migrations/001_initial_schema.up.sql` (verified). HelixConstitution clause numbers (§11.4.6 no-guessing, §11.4.74 catalogue-first, §11.4.61 ToC, §1 four-layer) are carried from corpus convention and are **UNVERIFIED** against the authoritative Constitution text (it is not present in this repository). Concrete rate-limit numbers, retention windows, and aggregate index/perf behavior are **UNVERIFIED** until set from MVP load tests. See §10 for the full register.
Fixed	Rev 1: first specification of the three deferred operational areas; maps each endpoint to the existing error envelope + RBAC roles; proposes the new `Repository` methods (`AppendAudit`, `ListAudit`, `TelemetryForDevice`, `TelemetryOverview`, and group CRUD) against the persistence seam in `server/internal/store/store.go`; reconciles the schema's `target_type IN ('all','group','device')` and the API's group narrowing.
Continuation	When these land in the build: wire the audit middleware on the protected route group **after** `requireRole` so only authorized mutating actions are logged; add the read routes behind `viewer`; add group CRUD behind `operator`/`admin`; bind concrete `limit`/window numbers from load tests; confirm the `database` brick exposes the proposed query methods (or implement in the pgx repository); add per-endpoint OpenTelemetry span names; extend `openapi.yaml` with these paths so the source-presence/artifact gates (§9) can diff them one-to-one.
Owner	Helix OTA control-plane team
Related	

Field	Value
	endpoints.md (parent REST spec — conventions, error model, RBAC); openapi.yaml; ../database/migrations/001_initial_schema.up.sql (audit_logs, device_groups, device_group_members, telemetry_events); ../../research/additions_synthesis.md (§8 G3/G4/G5, §9 items 3/4/5); ../security/signing_verification.md

table of contents

1. purpose_and_scope
2. inherited_conventions
3. rbac_role_map_for_these_endpoints
4. part_a_audit_subsystem
   - 4.1 audit_middleware_write_path
   - 4.2 append_audit_repo_method
   - 4.3 get_audit_query_endpoint
5. part_b_telemetry_reads
   - 5.1 get_device_telemetry_history
   - 5.2 get_telemetry_overview_aggregates
   - 5.3 telemetry_read_repo_methods
6. part_c_device_group_crud
   - 6.1 create_group
   - 6.2 list_and_read_groups
   - 6.3 update_and_delete_group
   - 6.4 membership_endpoints
   - 6.5 group_crud_repo_methods
7. error_model_mapping
8. status_code_summary
9. testing_four_layer
10. anti_bluff_unverified_register

1. purpose_and_scope

This document specifies three **operational** API areas that the synthesis routed out of the 1.0.0-MVP built surface and into the next operational/compliance phase (additions_synthesis.md §8 G3/G4/G5, §9 items 3/4/5). They are **deferred** — none are MVP blockers — but each has a canonical database table **already created** in database/migrations/001_initial_schema.up.sql, so the schema half is done and only the write/read path and routes are missing.

In scope:

- **(A) Audit** (G3 / §9.3) — a cross-cutting middleware that appends one `audit_logs` row per successful **mutating** admin/operator action, the `AppendAudit` repository method it calls, and a `GET /api/v1/audit` query endpoint for reading the trail.
- **(B) Telemetry reads** (G4 / §9.4) — the **read** side of telemetry: `GET /api/v1/devices/{deviceId}/telemetry` (per-device event history) and `GET /api/v1/telemetry/overview` (fleet aggregates). Telemetry **ingest** (`POST /api/v1/client/telemetry`, `endpoints.md §12.2`) is already built; this adds the read/analytics surface over the same `telemetry_events` rows.
- **(C) Device groups** (G5 / §9.5) — full CRUD over `device_groups` plus membership add/remove/list over `device_group_members`, so an operator can build the cohorts that `POST /api/v1/deployments` (`endpoints.md §11`) already narrows by `group`.

Out of scope: the staged-rollout engine (G7, `1.0.1-staged-rollout/`), the React dashboard (G6, `dashboard/`), mandatory-update/deadline semantics (G8), and tamper-event/rollout-halt (G9). These endpoints are the data **the dashboard consumes**, but the dashboard UI itself is a separate spec.

This spec is **subordinate to** `endpoints.md`: everything in its §2 (conventions), §3 (transport/compression), §4 (auth/RBAC), §5 (rate limiting), and §6 (error model) applies unchanged. Only the deltas are restated here.

2. inherited_conventions

Inherited verbatim from `endpoints.md` (not re-specified):

- **Base path** `/api/v1`; media type `application/json; charset=utf-8`; RFC-3339 UTC timestamps; opaque server-issued string ids (UUIDs in the schema).
- **Pagination** (`endpoints.md §2`): list endpoints accept `?limit` (default 50, max 200) and `?cursor` (opaque); responses carry `next_cursor` (null when exhausted). The audit and telemetry-history lists below use this pattern unchanged.
- **Transport/compression** (`endpoints.md §3`): all responses here are control-plane JSON, so they participate in `br` → `gzip` → `identity` negotiation with `Vary: Accept-Encoding`. None of these endpoints touch the artifact-byte path.
- **Auth header** `Authorization: Bearer <JWT>`; `authMiddleware` verifies, `requireRole` enforces (`endpoints.md §4`; built in `server/internal/api/middleware.go`).
- **Error envelope** (`endpoints.md §6`): the single `{ "error": { code, message, request_id, details[] } }` shape with the enumerated `code` set. This spec adds **no new error codes** — see §7 for the mapping of each failure to an existing code.
- **Correlation** `X-Request-Id` on every response; mirrored into `error.request_id`.

3. rbac_role_map_for_these_endpoints

Roles are the four from `endpoints.md §4.2` (`admin`, `operator`, `viewer`, `device`), enforced by the built `requireRole` middleware. Route → minimum role for the endpoints in this document:

Route	Method	Min role	Notes
`/audit`	`GET`	`admin`	Audit trail is sensitive (who-did-what); read restricted to `admin` (tighter than the `viewer`-readable status routes).
`/devices/{deviceId}/telemetry`	`GET`	`viewer`	Same read tier as `GET /devices/{deviceId}/status` (`endpoints.md §8.2`). A `device` token MAY read **only its own** id (own-resource rule, `endpoints.md §4.2`); a wrong-device `device` token → `403`.
`/telemetry/overview`	`GET`	`viewer`	Fleet-wide aggregate; no device-scoped access (a `device` token is `403`).
`/groups`	`POST`	`operator`	Cohort creation is an operator provisioning action.
`/groups`, `/groups/{groupId}`	`GET`	`viewer`	Read tier.
`/groups/{groupId}`	`PATCH`	`operator`	
`/groups/{groupId}`	`DELETE`	`admin`	Destructive; restricted to `admin`. Cascade detaches membership rows (FK `ON DELETE CASCADE`) but `deployments.target_group_id` is `ON DELETE SET NULL` (schema), so a deleted group does not orphan a deployment.
`/groups/{groupId}/members`	`GET`	`viewer`	List membership.
`/groups/{groupId}/members`	`POST`	`operator`	Add device(s) to a group.
`/groups/{groupId}/members/{deviceId}`	`DELETE`	`operator`	Remove one device.

admin inherits every operator/viewer capability (role hierarchy is expressed by listing all accepted roles in requireRole(...), as in the built router server.go). Authorization failures follow endpoints.md §4.2: missing/invalid token → 401 UNAUTHENTICATED; valid token lacking the role or crossing device ownership → 403 FORBIDDEN.

4. part_a_audit_subsystem

Gap **G3** (additions_synthesis.md §8 / §9.3): "Spec an audit middleware + AppendAudit repo method." The audit_logs table already exists (schema lines 367–384) with columns id, user_id (nullable, ON DELETE SET NULL so the record outlives the actor), action, resource_type, resource_id, details (JSONB), ip_address (INET), user_agent, created_at, and four indexes (user, action, (resource_type, resource_id), created_at).

4.1 audit_middleware_write_path

A Gin middleware (auditMiddleware) appended to the **protected** route group records every **successful mutating** admin/operator action.

- **Placement (ordering is load-bearing):** it runs **after** authMiddleware and **after** requireRole, so (a) the principal's sub is available from the verified claims, and (b) a request rejected by RBAC (401/403) is **not** audited as an action (it never ran). It writes **after** the handler completes (c.Next() then inspect c.Writer.Status()), so only an action that actually succeeded is logged.
- **Which requests are audited:** mutating methods only — POST, PUT, PATCH, DELETE — and only when the response status is a success (2xx). Read methods (GET) and failed mutations are **not** written (failed mutations are already visible in request logs/metrics; auditing them would let a probing attacker flood the trail). The GET /audit read endpoint (§4.3) is itself **not** audited (no infinite self-reference; reads are not actions).
- **Row mapping** (one audit_logs row per audited request):
  - user_id ← the users.id for the authenticated principal. The token sub (Claims.Subject) is the device/user subject; for human operators it resolves to a users row. If the subject does not resolve to a users row (e.g. a provisioning token), user_id is left NULL (the column is nullable by design) and the subject is preserved in details.actor_subject.
  - action ← a stable verb derived from method + route, SCREAMING_SNAKE_CASE, e.g. ARTIFACT_UPLOAD, RELEASE_CREATE, DEPLOYMENT_CREATE, DEVICE_REGISTER, GROUP_CREATE, GROUP_UPDATE, GROUP_DELETE, GROUP_MEMBER_ADD, GROUP_MEMBER_REMOVE. (Indexed by idx_audit_logs_action.)
  - resource_type ← the noun (artifact, release, deployment, device, group, group_member).
  - resource_id ← the affected resource id (from the path param, or the created id returned in the 2xx body). NULL when not resolvable.
  - details ← a JSONB redacted summary: never the full request body, **never** secrets, passwords, tokens, signatures, or password hashes (the recovery/ redaction discipline from endpoints.md §6 applies). Safe fields only (e.g. {"version":"1.1.0","strategy":"all-targets"}).

    - `ip_address` ← `c.ClientIP()` (respecting the trusted-proxy config); `user_agent` ← the `User-Agent` header (truncated to a safe length).
    - `created_at` ← DB default `now()`.
- **Failure isolation:** an audit-write failure MUST NOT fail the user's already-successful request (the action happened; the response was already chosen). The write is best-effort with respect to the response status, but a write error is logged to observability and surfaced as a monitored metric (a persistently failing audit sink is an operational alert via `Herald`, not a `500` to the caller). (UNVERIFIED: whether compliance posture requires **synchronous, fail-closed** auditing — if so, this flips to writing before the `2xx` is flushed and failing the request on write error; left as a deferred-phase decision, see §10.)
- **Reuse (catalogue-first, §11.4.74 UNVERIFIED):** the middleware is wired in the existing `middleware.go` alongside `authMiddleware`/`requireRole`; persistence goes through the `Repository` seam (§4.2), not direct SQL in the handler. No new transport or auth logic is hand-rolled.

4.2 append_audit_repo_method

New method on the `store.Repository` interface (`server/internal/store/store.go`), plus a domain struct. No transport types cross the seam (per the store package doc comment).

```
// AuditEntry is one persisted admin/operator action (audit_logs). Maps 1:1 to
// the schema columns; UserID is empty when the actor does not resolve to a
// users row (the column is nullable by design).
type AuditEntry struct {
    ID           string            // audit_logs.id (server-minted)
    UserID       string            // users.id; empty => NULL
    ActorSubject string            // token sub, preserved in details when UserID empty
    Action       string            // SCREAMING_SNAKE_CASE verb
    ResourceType string            // artifact|release|deployment|device|group|group_member
    ResourceID   string            // affected id; empty => NULL
    Details      map[string]any    // redacted JSONB summary (no secrets)
    IPAddress    string            // INET (string form)
    UserAgent    string
    CreatedAt    time.Time
}

// AuditFilter narrows a GET /audit query (§4.3). Zero-value fields are ignored.
type AuditFilter struct {
    Actor        string    // users.id OR token subject
    Action       string    // exact action verb
    ResourceType string
    ResourceID   string
    Since        time.Time // created_at >= Since
    Until        time.Time // created_at <  Until
    Limit        int       // pagination (default 50, max 200)
```

```
    Cursor       string    // opaque
}

// Append-only audit write (G3). Best-effort relative to the
        request (§4.1):
// a write error is returned to the middleware, which logs/metrics
        it but does
// NOT fail the user's already-successful action.
AppendAudit(ctx context.Context, e AuditEntry) error

// Paginated, filtered audit read backing GET /audit (§4.3).
        Newest-first.
ListAudit(ctx context.Context, f AuditFilter) (entries
        []AuditEntry, nextCursor string, err error)
```

The in-memory `MemoryRepository` implements both (append to a slice, filter+page in memory) so the api seam stays testable without PostgreSQL; the pgx implementation issues an `INSERT` and a filtered, index-backed `SELECT ... ORDER BY created_at DESC` (the `idx_audit_logs_*` indexes cover the filter columns).

4.3 get_audit_query_endpoint

`GET /api/v1/audit` — read the audit trail.

- **Auth:** `admin` (§3). A `viewer`/`operator` token → `403`.
- **Query parameters** (all optional; combine with AND): `?actor=` (users.id or subject), `?action=`, `?resource_type=`, `?resource_id=`, `?since=` (RFC-3339), `?until=` (RFC-3339), plus the inherited `?limit` / `?cursor`.
- **Response 200** (`AuditList`): newest-first page.

```
{
  "items": [
    {
      "id": "9c2f...uuid",
      "actor": { "user_id": "1a2b...uuid", "subject":
        "admin@example.com" },
      "action": "DEPLOYMENT_CREATE",
      "resource_type": "deployment",
      "resource_id": "d12b...uuid",
      "details": { "release_id": "r77e...uuid", "strategy": "all-
        targets" },
      "ip_address": "203.0.113.7",
      "user_agent": "helix-dashboard/1.0",
      "created_at": "2026-06-08T10:15:00Z"
    }
  ],
  "next_cursor": null
}
```

- **Status codes:** `200` OK; `400 VALIDATION_FAILED` (malformed `since`/`until`/`limit`); `401 UNAUTHENTICATED`; `403 FORBIDDEN` (not `admin`); `429 RATE_LIMITED`.
- **Immutability:** the audit trail is **read-only over the API** — there is no `POST`, `PATCH`, or `DELETE /audit`. Rows are written only by the middleware (§4.1).

Retention/rotation is an operational (DB-side) concern, **UNVERIFIED** here (no retention window is asserted — §10).

5. part_b_telemetry_reads

Gap **G4** (`additions_synthesis.md` §8 / §9.4): "Spec read endpoints in `endpoints.md`; dashboard/monitoring phase." Ingest is already built (`POST /api/v1/client/telemetry`, `endpoints.md §12.2`, schema-validated via `ota-telemetry-schema`); the `telemetry_events` table exists (schema lines 329–361) with a canonical 8-value `event_type` CHECK (`download_started`, `download_complete`, `installing`, `installed`, `verifying`, `success`, `failure`, `rollback`) and indexes on `device_id`, `deployment_id`, `event_type`, `created_at`. This part adds **only reads** — no new ingest, no schema change.

5.1 get_device_telemetry_history

`GET /api/v1/devices/{deviceId}/telemetry` — the per-device event history.

- **Auth:** `viewer` (admin/operator/viewer). A `device` token MAY read **only its own** `deviceId` (own-resource rule, `endpoints.md §4.2`); a `device` token for another id → `403`.
- **Query parameters** (optional): `?event_type=` (one of the 8 canonical values), `?since=` / `?until=` (RFC-3339), `?deployment_id=` (narrow to one deployment), plus inherited `?limit` / `?cursor`. Ordered **newest-first** by `created_at`.
- **Response 200** (`TelemetryHistory`):

```
{
  "device_id": "8f3a...uuid",
  "items": [
    {
      "id": "e01a...uuid",
      "deployment_id": "d12b...uuid",
      "event_type": "failure",
      "version": "1.1.0",
      "success": false,
      "error_code": "PAYLOAD_VERIFICATION_FAILED",
      "error_message": "FILE_HASH mismatch",
      "duration_ms": 41230,
      "bytes_transferred": 379074366,
      "created_at": "2026-06-08T10:20:00Z"
    }
  ],
  "next_cursor": null
}
```

Fields map directly to `telemetry_events` columns; `version` is read from the row `metadata` JSONB (ingest stores the reported `version` there per the telemetry codec). The read surface does **not** invent fields not present in the ingested rows.

- **Status codes:** `200` OK; `400 VALIDATION_FAILED` (bad `event_type`/time/limit); `401`/`403`; `404 NOT_FOUND` (no such device); `429 RATE_LIMITED`.

5.2 get_telemetry_overview_aggregates

`GET /api/v1/telemetry/overview` — fleet-wide / per-deployment aggregate counters for the monitoring dashboard.

- **Auth:** `viewer`. A `device` token → `403` (no device-scoped aggregate).
- **Query parameters** (optional): `?deployment_id=` (scope the aggregate to one deployment; omitted ⇒ whole fleet), `?since=` / `?until=` (RFC-3339 window over `created_at`), `?os=` (narrow by device os). **No pagination** — this is a single aggregate object.
- **Response 200** (`TelemetryOverview`): counts grouped by terminal/lifecycle state, derived with the **latest-event-per-device** rule already used by `deriveProgress` (`handlers_deployment.go`), so the overview and the per-deployment progress (`endpoints.md §11.2`) agree by construction.


```
{
  "scope": { "deployment_id": null, "os": "android", "since":
        null, "until": null },
  "devices_total": 1240,
  "devices_reporting": 1190,
  "by_state": {
    "pending": 50,
    "downloading": 30,
    "installing": 12,
    "installed": 8,
    "verifying": 4,
    "succeeded": 1100,
    "failed": 36,
    "rolled_back": 0
  },
  "event_counts": {
    "download_started": 1188, "download_complete": 1160,
    "installing": 1152, "installed": 1140, "verifying": 1136,
    "success": 1100, "failure": 36, "rollback": 0
  },
  "failure_rate": 0.030,
  "generated_at": "2026-06-08T10:25:00Z"
}
```


- `by_state` uses the **latest event per device** (one device counted once); `event_counts` is the **raw event tally** over the window (a device contributes multiple events). The two are intentionally different denominators and are labelled as such.
- `failure_rate` = `by_state.failed / devices_reporting` (0 when `devices_reporting` is 0; never divides by zero). `rolled_back` is sourced from the `rollback` event (MVP records the event; the full rollback engine is `1.0.2`, `additions_synthesis.md` §11).
- **Status codes:** `200` OK; `400 VALIDATION_FAILED` (bad scope/time); `401`/`403`; `404 NOT_FOUND` (when `?deployment_id=` names a non-existent deployment); `429`.
- **Performance/UNVERIFIED:** the aggregate is computed over `telemetry_events` filtered by the existing indexes; whether a materialized rollup is needed at fleet scale is **UNVERIFIED** until measured (no NFR number asserted — `additions_synthesis.md` §8 G12, §10 below).

5.3 telemetry_read_repo_methods

New read methods on `store.Repository` (the built interface already has `AppendTelemetry` + `TelemetryForDeployment`; these add the device-history and overview reads):

```
// TelemetryFilter narrows a per-device history query (§5.1).
type TelemetryFilter struct {
    DeviceID     string
    EventType
        string    // one of the 8 canonical values; empty => all
    DeploymentID string
    Since, Until time.Time
    Limit        int
    Cursor       string
}

// OverviewFilter scopes the aggregate (§5.2). All fields
        optional.
type OverviewFilter struct {
    DeploymentID string
    OSType       otaprotocol.OSType
    Since, Until time.Time
}

// TelemetryOverview is the aggregate result (§5.2) — latest-
        event-per-device
// state counts plus raw event tallies over the window.
type TelemetryOverview struct {
    DevicesTotal     int
    DevicesReporting int
    ByState          map[string]int // pending|downloading|...|
        rolled_back
    EventCounts      map[string]int // the 8 canonical event types
    GeneratedAt      time.Time
}

// Per-device telemetry history backing GET /devices/{id}/
        telemetry (§5.1),
// newest-first, paginated.
TelemetryForDevice(ctx context.Context, f TelemetryFilter) (recs
        []TelemetryRecord, nextCursor string, err error)

// Fleet/deployment aggregate backing GET /telemetry/overview
        (§5.2).
TelemetryOverviewAgg(ctx context.Context, f OverviewFilter)
        (TelemetryOverview, error)
```

The in-memory implementation iterates its event slice (filter + latest-per-device fold), reusing the same logic as the built `deriveProgress`; the pgx implementation uses the `created_at` / `device_id` / `event_type` indexes and a `DISTINCT ON (device_id) ... ORDER BY device_id, created_at DESC` for the latest-per-device fold. (UNVERIFIED: that the `database` brick exposes a helper for `DISTINCT ON` / windowed reads — otherwise implemented directly in the pgx repo.)

6. part_c_device_group_crud

Gap **G5** (`additions_synthesis.md` §8 / §9.5): "Spec group endpoints + repo methods." The `device_groups` table (schema 109–116: `id`, `name` UNIQUE, `description`, `filter_criteria` JSONB, `created_at`) and `device_group_members` (118–130: composite PK `(group_id, device_id)`, both FKs `ON DELETE CASCADE`, `added_at`) already exist. `deployments.target_group_id` already references `device_groups(id)` (`ON DELETE SET NULL`), and `POST /api/v1/deployments` already accepts a `group` narrowing (`endpoints.md §11.1`); this part lets an operator **create and populate** those cohorts.

A group is **static or dynamic**: static membership lives in `device_group_members`; a dynamic selector lives in `filter_criteria` (JSONB). MVP-of-this-phase supports **static membership fully**; dynamic `filter_criteria` is **stored and returned but its evaluation is UNVERIFIED** (the selector language is not yet specified — §10).

6.1 create_group

`POST /api/v1/groups`

- **Auth:** `operator`. Accepts optional `Idempotency-Key` (`endpoints.md §2`).
- **Request body** (`GroupCreate`):

```
{
  "name": "field-fleet-a",
  "description": "EU-west production Orange Pi 5 Max fleet",
  "filter_criteria": { "metadata.region": "eu-west" }
}
```

`name` is required and must be unique (DB `device_groups_name_uniq`). `filter_criteria` is optional JSONB (null ⇒ static-only group).

- **Response 201** (`Group`):

```
{
  "group_id": "g55a...uuid",
  "name": "field-fleet-a",
  "description": "EU-west production Orange Pi 5 Max fleet",
  "filter_criteria": { "metadata.region": "eu-west" },
  "member_count": 0,
  "created_at": "2026-06-08T10:30:00Z"
}
```

- **Status codes:** `201`; `200` (idempotent replay); `400 VALIDATION_FAILED` (missing/blank `name`); `401`/`403`; `409 CONFLICT` (name already exists); `429`.

6.2 list_and_read_groups

- `GET /api/v1/groups` — **Auth** `viewer`; paginated (`endpoints.md §2`); optional `?name=` prefix filter. Response `GroupList { items: [Group], next_cursor }`.

- `GET /api/v1/groups/{groupId}` — **Auth** `viewer`; response `Group` (with current `member_count`); `404 NOT_FOUND` if absent.

6.3 update_and_delete_group

- `PATCH /api/v1/groups/{groupId}` — **Auth** `operator`. Partial update of `description` and/or `filter_criteria` (and `name`, subject to the uniqueness constraint). Body is a sparse `GroupUpdate`; only present fields change. Response `200` `Group`. `404` if absent; `409 CONFLICT` if a `name` change collides.
- `DELETE /api/v1/groups/{groupId}` — **Auth** `admin` (destructive, §3). Deletes the group; membership rows cascade-delete (FK), and any `deployments.target_group_id` referencing it is set `NULL` (FK), so no deployment is orphaned. Response `204 No Content`. `404` if absent. (UNVERIFIED policy decision: whether an **active** deployment targeting the group should **block** deletion with `409` instead of `SET NULL` — left as a deferred-phase choice, §10.)

6.4 membership_endpoints

Membership operates on `device_group_members`. Device ids in request bodies are the external `device_id` (the registry identity, `endpoints.md §8.1`), resolved to the internal `devices.id` for the FK.

- `GET /api/v1/groups/{groupId}/members` — **Auth** `viewer`; paginated. Response `GroupMemberList`:

```
{
  "group_id": "g55a...uuid",
  "items": [
    { "device_id": "8f3a...uuid", "added_at":
        "2026-06-08T10:31:00Z" }
  ],
  "next_cursor": null
}
```

- `POST /api/v1/groups/{groupId}/members` — **Auth** `operator`. Add one or more devices. Body `GroupMemberAdd`:

```
{ "device_ids": ["8f3a...uuid", "7b2c...uuid"] }
```

Response `200` `GroupMemberAddResult`
`{ "added": 2, "already_member": 0, "not_found": [] }`. Adding an already-present `(group_id, device_id)` is **idempotent** (counted in `already_member`, not an error — the composite PK makes a re-add a no-op via upsert). A `device_id` that does not resolve to a registered device is returned in `not_found` and the request still succeeds with the resolvable ones (partial success is reported, not a hard `404` for the whole batch). `404 NOT_FOUND` is reserved for a missing **group**.

- `DELETE /api/v1/groups/{groupId}/members/{deviceId}` — **Auth** `operator`. Remove one device. Response `204 No Content`. `404 NOT_FOUND` if the group does not exist; removing a device that is not a member is **idempotent** (`204`, not `404` — DELETE is idempotent on absence).

6.5 group_crud_repo_methods

New methods + structs on `store.Repository`:

```
// Group is a device cohort (device_groups). MemberCount is
        derived, not stored.
type Group struct {
    GroupID        string
    Name           string
    Description    string
    FilterCriteria
        map[string]any // device_groups.filter_criteria (nullable)
    MemberCount    int
    CreatedAt      time.Time
}

// GroupMember is one static membership row
        (device_group_members).
type GroupMember struct {
    GroupID   string
    DeviceID  string    // external device_id
    AddedAt   time.Time
}

// GroupFilter narrows GET /groups (§6.2).
type GroupFilter struct {
    NamePrefix string
    Limit      int
    Cursor     string
}

// Group CRUD (§6.1–6.3).
CreateGroup(ctx context.Context, g Group) error
GetGroup(ctx context.Context, groupID string) (Group, error)
ListGroups(ctx context.Context, f GroupFilter) (groups []Group,
        nextCursor string, err error)
UpdateGroup(ctx context.Context, g Group) error // sparse-applied
        by the handler
DeleteGroup(ctx context.Context, groupID string) error

// Membership (§6.4). AddGroupMembers is idempotent on the
        composite PK and
// returns the per-device outcome; resolution of device_id ->
        devices.id happens
// in the repo. RemoveGroupMember is idempotent on absence.
AddGroupMembers(ctx context.Context, groupID string, deviceIDs
        []string) (added, alreadyMember int, notFound []string,
        err error)
ListGroupMembers(ctx context.Context, groupID string, limit int,
        cursor string) (members []GroupMember, nextCursor string,
        err error)
RemoveGroupMember(ctx context.Context, groupID, deviceID string)
        error
```

`CreateGroup`/`UpdateGroup` return `ErrConflict` on the unique-name violation (mapped to `409 CONFLICT` by the handler, matching the built `ErrConflict→409` convention). `GetGroup`/ membership methods return `ErrNotFound` for a missing group (→ `404`). The in-memory repo implements all of these so the routes are testable without PostgreSQL.

7. error_model_mapping

No new error codes are introduced (per `endpoints.md §6` enumerated set, mirrored in `server/internal/api/errors.go`). Each failure maps to an existing code:

Condition	HTTP	`code`
Missing/invalid/expired bearer token	401	`UNAUTHENTICATED`
Authenticated but lacks the route's role, or `device` token crossing ownership (§5.1)	403	`FORBIDDEN`
Malformed body, bad `since`/`until`/`limit`, blank `name`, bad `event_type`	400	`VALIDATION_FAILED` (+ `details[]`)
Group / device / deployment id not found	404	`NOT_FOUND`
Duplicate group `name` (create or rename)	409	`CONFLICT`
Rate limit exceeded	429	`RATE_LIMITED` (+ `Retry-After`)
Unhandled server fault (via `recovery` brick)	500	`INTERNAL` (no disclosure)

The audit-write failure path (§4.1) is **not** a caller-visible error — it does not map to a `code`; it is logged/metric'd and (if persistent) alerted via `Herald`. The error envelope shape and `respondError`/`respondValidation` helpers (`errors.go`) are reused unchanged.

8. status_code_summary

Code	Meaning in these endpoints	Where
200	OK (audit list, telemetry read/overview, group read/list/update, member add)	§4.3, §5.1, §5.2, §6.2, §6.3, §6.4
201	Created (group)	§6.1
204	No Content (group delete, member remove)	§6.3, §6.4

Code	Meaning in these endpoints	Where
400	`VALIDATION_FAILED`	all write/query routes
401	`UNAUTHENTICATED`	all routes
403	`FORBIDDEN` (role/ ownership)	all routes
404	`NOT_FOUND`	single-resource reads, member ops, scoped overview
409	`CONFLICT` (duplicate group name)	§6.1, §6.3
429	`RATE_LIMITED`	all rate-limited routes (`endpoints.md §5`)
500	`INTERNAL` (no disclosure)	any (via `recovery` brick)

9. testing_four_layer

Per `endpoints.md §14` / master §13 / Constitution §1 (UNVERIFIED clause), every change ships four layers with no-bluff positive evidence. The audit write path is treated as **safety/ compliance-critical** and targets the ≥90% floor.

1. **Source-presence gate.** Each route here is registered on the Gin router under `/api/v1/...` (`/audit`, `/devices/:deviceId/telemetry`, `/telemetry/overview`, `/groups`, `/groups/:groupId`, `/groups/:groupId/members`, `/groups/:groupId/members/:deviceId`); each new `Repository` method (`AppendAudit`, `ListAudit`, `TelemetryForDevice`, `TelemetryOverviewAgg`, `CreateGroup` …) is declared and implemented by the in-memory repo; `openapi.yaml` gains the matching paths and they diff one-to-one against the route table.
2. **Artifact gate (bytes shipped).** Boot the server; assert the live route table exposes these paths with the specified methods and roles, and that `auditMiddleware` is mounted **after** `requireRole` on the protected group (ordering assertion — a mutating `2xx` produces exactly one `audit_logs` row; a `403` produces **zero**).
3. **Runtime / integration.** Against a running monolith + PostgreSQL: `operator` creates a group → adds members → creates an all-targets deployment narrowed by that group → device reports telemetry → `GET /devices/{id}/telemetry` returns the history → `GET /telemetry/overview` aggregate matches the per-deployment progress (`endpoints.md §11.2`) → `GET /audit` (as `admin`) shows the `GROUP_CREATE` + `DEPLOYMENT_CREATE` rows and **no** row for the `GET`s. Negative: `viewer` hitting `/audit` → `403`; `device` token reading another device's telemetry → `403`; duplicate group name → `409`; deleting a group `SET NULL`s a referencing deployment without orphaning it.
4. **Mutation meta-test (PASS→FAIL on negation).** Mutate and assert the suite flips: move `auditMiddleware` **before** `requireRole` (must fail the "403 writes zero audit rows" test); make the audit write include the request body verbatim (must fail the "no-secrets in details" redaction test); change the overview to count all events instead of latest-per-device (must fail the "overview agrees with deployment progress" test); drop the `device` own-ownership check on `/`

`devices/{id}/telemetry` (must fail the `403` cross-device test). A mutation that breaks no test exposes a coverage hole and is itself a defect.

10. anti_bluff_unverified_register

Per Constitution §11.4.6 (no-guessing) / §7.1 (no-bluff), the following MUST NOT be propagated as fact:

- **Not built yet.** Audit middleware, the `GET /audit` endpoint, telemetry read endpoints, and group CRUD routes do **not** exist in `server/internal/api` as of this revision (`additions_synthesis.md` §8 G3/G4/G5 — confirmed against the built handler set this pass). The backing tables (`audit_logs`, `device_groups`, `device_group_members`, `telemetry_events`) **do** exist in `001_initial_schema.up.sql` (verified).
- **Proposed, not merged.** Every `Repository` method in §4.2/§5.3/§6.5 is a **proposal** against `store.go`; the interface today carries only `AppendTelemetry`/`TelemetryForDeployment` on the telemetry side and has **no** audit or group methods (verified).
- **Audit posture.** Whether compliance requires **synchronous, fail-closed** auditing (write before the `2xx`, fail the request on write error) vs. the best-effort post-response write specified in §4.1 is **UNVERIFIED** — a deferred-phase decision.
- **Group deletion vs active deployment.** Whether an active deployment targeting a group should **block** delete (`409`) instead of the schema's `SET NULL` is **UNVERIFIED** (§6.3).
- **Dynamic `filter_criteria`.** The selector language for dynamic groups is **not specified**; `filter_criteria` is stored/returned but its **evaluation** is UNVERIFIED — static membership is the supported path this phase (§6).
- **Aggregate performance.** Whether `GET /telemetry/overview` needs a materialized rollup at fleet scale is **UNVERIFIED** until measured; no NFR number (10k devices, <100ms, 99.9%) is asserted (`additions_synthesis.md` §8 G12).
- **Rate limits / retention.** Concrete `limit` values and any audit/telemetry retention window are **UNVERIFIED** until set from MVP load tests (`endpoints.md §5` continuation).
- **Catalogue brick surfaces.** That the `database` brick exposes helpers for `DISTINCT ON` / windowed reads, and that `Herald` is the audit-failure alerting path, are reused **assumptions** (UNVERIFIED — `submodule_reuse_map.md` §5; §11.4.74).
- **Constitution clause numbers** (§11.4.6, §11.4.74, §11.4.61, §1) are carried from corpus convention and remain **UNVERIFIED** against the authoritative Constitution text (not present in this repository).